

Lecture: Quick Sort Algorithm — From Zero to Advanced

1. Introduction

Quick Sort is one of the most efficient and widely used **divide-and-conquer sorting algorithms**.

It works by selecting a **pivot element**, partitioning the array into two subarrays (elements less than pivot, and elements greater than pivot), and recursively sorting the subarrays.

Definition:

Quick Sort repeatedly partitions an array based on a pivot, sorting elements into left (smaller) and right (larger) groups, and then recursively sorts the partitions.

Real-life analogy:

Imagine organizing books by choosing one book as a reference (pivot), placing all smaller books to the left and larger books to the right, and then repeating the process for each side until sorted.

2. Key Characteristics

- **Type:** Divide-and-Conquer Sorting
- **Input Requirement:** Works on any list (sorted or unsorted)
- **Direction:** Partition based on pivot, recursive sorting of subarrays
- **Time Complexity:**
 - Best: $O(n \log n)$
 - Average: $O(n \log n)$
 - Worst: $O(n^2)$ (if pivot is always smallest/largest element)
- **Space Complexity:** $O(\log n)$ (for recursion stack)
- **Stability:** Not stable by default (can be modified)

3. Step-by-Step Working

1. Choose a pivot element (first, last, middle, or random).
2. Partition the array so that all smaller elements are on the left and all greater elements are on the right of the pivot.
3. Recursively apply Quick Sort to the left and right subarrays.

4. Combine results → sorted array.

4. Pseudocode

QuickSort(arr, low, high):

if low < high:

 pivot_index = Partition(arr, low, high)

 QuickSort(arr, low, pivot_index - 1)

 QuickSort(arr, pivot_index + 1, high)

Partition(arr, low, high):

 pivot = arr[high]

 i = low - 1

 for j = low to high-1:

 if arr[j] <= pivot:

 i = i + 1

 swap(arr[i], arr[j])

 swap(arr[i+1], arr[high])

 return i + 1

5. Complexity Analysis

- **Best Case:** $O(n \log n)$ → balanced partitions
- **Worst Case:** $O(n^2)$ → unbalanced partitions (e.g., sorted input with poor pivot choice)
- **Average Case:** $O(n \log n)$
- **Space Complexity:** $O(\log n)$ (recursion stack)
- **Stability:** Not stable by default

6. Handling Edge Cases

1. **Empty list** → already sorted.

2. **Single element list** → already sorted.
3. **All elements identical** → works fine but performance may degrade.
4. **Already sorted array** → worst case $O(n^2)$ if pivot chosen poorly.

7. Variations of Quick Sort

- **Randomized Quick Sort** → Pivot chosen randomly to avoid worst-case.
- **Median-of-Three Quick Sort** → Pivot chosen as median of first, middle, last elements.
- **Iterative Quick Sort** → Eliminates recursion with explicit stack.
- **Dual-Pivot Quick Sort** → Uses two pivots to partition into three sections (used in Java's `Arrays.sort()`).

8. When to Use Quick Sort

Best suited for:

- Large datasets.
- When average-case performance matters more than guaranteed worst case.
- In-memory sorting.

Avoid if:

- Worst-case $O(n^2)$ performance is risky (use Merge Sort instead).
- Stability is required.

9. Advanced Usage

- Widely used in **system libraries** (Python, C, Java often use hybrid versions).
- Forms the basis of **hybrid algorithms** (e.g., Introsort).
- Optimized for **in-memory sorting** because of low overhead and cache efficiency.

10. Real-World Applications

- **Databases:** Sorting records quickly.
- **Programming libraries:** Many built-in sorting functions are based on Quick Sort.
- **Big data systems:** Randomized Quick Sort for performance balance.
- **Operating systems:** Used in process scheduling and resource allocation.

11. Summary Table

Feature	Quick Sort
Works on sorted?	Yes (but may degrade)
Works on unsorted?	Yes
Time Complexity	Best: $O(n \log n)$, Worst: $O(n^2)$, Avg: $O(n \log n)$
Space Complexity	$O(\log n)$
Stable?	No (by default)

12. Practice Tasks

1. Implement Quick Sort recursively in Python.
2. Implement **Randomized Quick Sort**.
3. Modify Quick Sort to sort in descending order.
4. Compare Quick Sort and Merge Sort for datasets of size 10^5 .
5. Implement **Iterative Quick Sort** using a stack.

13. Closing Thought

Quick Sort is one of the most efficient sorting algorithms in practice.

Although its worst case is $O(n^2)$, careful pivot selection makes it perform near $O(n \log n)$ in most cases.

It is fast, memory-efficient, and widely used in **real-world applications**.

Mastering Quick Sort is essential for understanding advanced sorting and **hybrid algorithms**.